

成 绩	
教师签字	

通 信 工 程 学 院

实 验 报 告

()

实验题目： 生命游戏

班级： 202202 姓名： 孟德昭 学号： 20220219

实验日期： 2023 年 3 月 6 日

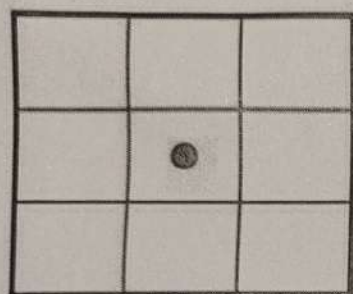
实验任务：

请写一个模拟生命游戏的程序；所谓生命游戏，请看下面的说明。

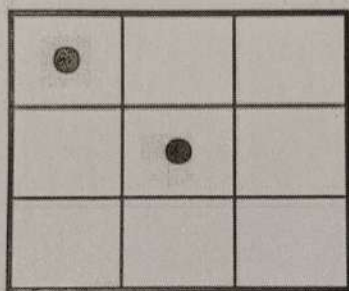
【说明】

生命游戏是一个很简单，但却是很有趣的程序习题。在一个四周都可以延伸到无限的棋盘上的某个格子中会有一个有机体。每一个有机体在时间 t 时，会依照环绕着它的 8 个邻居的特性而决定在时间 $t+1$ 时是否能生存下去。如果某一格在时间 t 时：

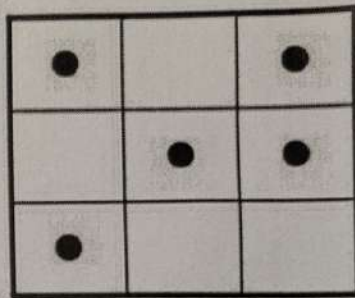
(1) 有一个有机体，但是它的邻居少于或等于 1 个，或者是大于 3 个，那就会因为不够稠密或太过稠密，这个有机体在时间 $t+1$ 时就会死亡；换言之，在 $t+1$ 时间，那一格中不会存在有机体。下面就是几个在时间 $t+1$ 时会死亡的例子，如图 1 所示。



太 少



太 少



太 多

图 1

(2) 有机体在该处，当只有 2 个或 3 个邻居时，就可以继续生存。下面就是几个可以生存的例子，如图 2 所示。

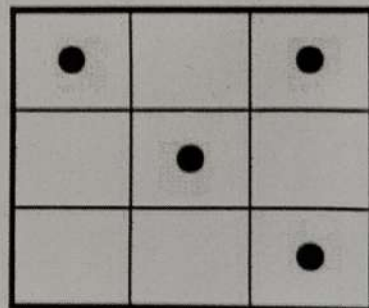
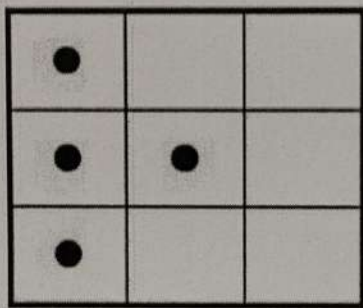
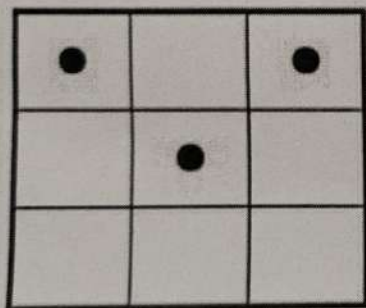


图 2

(3) 如果没有有机体，而那一格恰好有 3 个邻居时，当时间为 $t+1$ 时，那一格就会生出一个有机体。下面就是几个可以从“无”到“有”的例子，如图 3 所示。

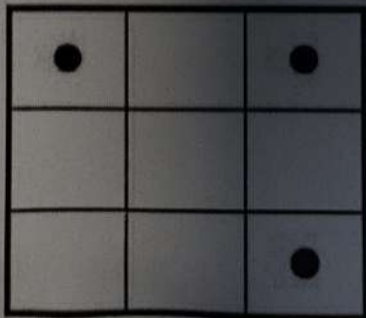
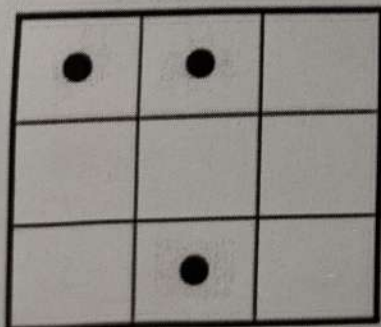


图 3

实验记录:

(4) 其他情形不会造成新的有机体出生。

程序要输入棋盘的大小, 并且输入最大的时间值, 比如 N , 然后输入一个在开始时的有机体分布情况, 最后显示出时间为 $2, 3, \dots, N$ 时棋盘上的状态。

一般而言, 最终的结果不外乎 4 种。第一种是不到时间 N 时, 所有有机体全都死光; 第二种是到了某个时间, 棋盘就不再发生变化, 生者生存, 这就叫做稳定状态; 第三种则是到了某个时间之后, 棋盘上的分布方式就产生循环, 也就是说, 棋盘上有某几种状况周而复始地以周期性方式出现; 第四, 也就是最后一种, 它不是上述三种情况之一。程序应该可以查出在时间 N 之前棋盘进入稳定状态, 或者是有机体全都死, 第三种情况是不容易找出来的, 为什么?

实!

```
#include<stdio.h>
void get(int array[][200],int ,int);
void put(int p[][200],int ,int);
void update(int c[][200],int ,int );
int judge(int d[][200],int ,int ,int);
void transform(int s[][200],int x,int y,int*);
int f[10000];//为使主函数和各个功能函数都能调用，将该数组定义为全局变量
int main ()
{
    int a[200][200],j,k,N,t,m=0; //先设置一个足够大的空间在形参中控制其具体大小
    printf("请输入棋盘的行和列:");
    scanf("%d%d",&j,&k);//输入数组行和列
    printf("请输入生存时间: ");
    scanf("%d",&N);
    printf("请初始化棋盘，生存输入 1，死亡输入 0\n");
    get(a,j,k);//读入数组中元素的值，实现初始化
    for(int i=0;i<j;i++)
        for(int q=0;q<k;q++)
        {
            if(a[i][q]!=0&&a[i][q]!=1)
            {
                printf("数据非法! ");
                return 0;
            }
        }
    //判断输入数据是否合法
    printf("第 1 秒的状态\n");
    put(a,j,k);//打印初始化时的状态
    printf("\n");
    transform(a,j,k,f);//将第一秒的数据放入 f[10000]中
    for(int i=2;i<=N;i++)
    {
        update(a,j,k); //更新数组
        printf("第%d 秒的状态\n",i);
        put(a,j,k); //打印每秒的状态
        printf("\n");
        transform(a,j,k,f+j*k*(i-1)); //把每次的数据读入一维数组
    }
    switch(judge(a,j,k,N))//利用 judge 函数的返回值确定棋盘状态
    {
        case 1:printf("最终全死光\n");break;
        case 2:printf("最终不再变化\n");break;
        case 3:printf("最终不断产生循环\n");break;
        default :printf("未达到三种情况\n");
    }
}
```


实验记录:

```
        return 0;
    }

    void get(int array[][200], int x, int y) //读入数组
    {
        int i=0, j=0, k;
        for(k=0; k<x*y; k++)
        {
            scanf("%d", &array[i][j++]);
            if(j==y) { i++; j=0; }
        }
    }

    void put(int p[][200], int x, int y) //输出某一秒的数组
    {
        int i, j;
        for(i=0; i<x; i++)
            for(j=0; j<y; j++)
            {
                printf("%d\t", p[i][j]);
                if(j==y-1) printf("\n");
            }
    }

    void update(int c[][200], int m, int n) //为当前一秒的数组赋新值
    {
        int i=0, j=0, e=0, f=0;
        int aa[m][n];
        for(e=0; e<m; e++)
            for(f=0; f<n; f++)
            {
                int geshu=0;
                if(c[e-1][f-1]==1&&e>0&&f>0) geshu++;
                if(c[e-1][f]==1&&e>0) geshu++;
                if(c[e-1][f+1]==1&&e>0&&f+1<n) geshu++;
                if(c[e][f-1]==1&&f>0) geshu++;
                if(c[e][f+1]==1&&f+1<n) geshu++;
                if(c[e+1][f-1]==1&&e+1<m&&f>0) geshu++;
                if(c[e+1][f]==1&&e+1<m) geshu++;
                if(c[e+1][f+1]==1&&e+1<m&&f+1<n) geshu++;
                aa[e][f]=geshu; //将该秒每个生命体相邻元素个数读入数组
            }
        for(e=0; e<m; e++)
            for(f=0; f<n; f++)
            {
                if(c[e][f]==1)
                {
                    if(aa[e][f]==0||aa[e][f]==1||aa[e][f]>3) c[e][f]=0;
                }
            }
    }
}
```

实验

```

        } //上一秒存活状态下判定, 为这一秒元素赋新值
        if(c[e][f]==0)
        {
            if(aa[e][f]==3) c[e][f]=1;
        } //上一秒死亡状态下判定, 为这一秒元素赋新值
    }
}

void transform(int s[][200],int x,int y,int*p) //把每一秒的数据存入一维数组
{
    for(int i=0;i<x;i++)
        for(int j=0;j<y;j++)
        {
            *p=s[i][j];
            p++;
        }
}

int judge(int d[][200],int x,int y,int q) //判断结果
{
    int i,j,c=0,r;
    for(i=0;i<x;i++)
        for(j=0;j<y;j++)
        {
            if(d[i][j]!=0) c++;
        }
    if(c==0) return 1; //返回 1, 全死亡
    c=0; //如果未全死亡, c 的值已改变, 重新赋值 0, 否则影响接下来判断
    for(r=0;r<q;r++)
    {
        for(int i=0;i<x*y&&r<q-1;i++)
        {
            if(f[i+x*y*r]==f[i+x*y*(r+1)]) c++;
            else {c=0;break;}
        }
        if(c==x*y) return 2; //返回 2, 处于静止状态且不全为 0
    }
    for(r=0;r<q;r++)
    {
        int s=2;
        for(int i=0;i<x*y&&r<q-1&&(r+s<=q);i++,s++)
        {
            if(f[i+x*y*r]==f[i+x*y*(r+s)])
                return 3; //返回 3, 产生了周期性
        }
    }
    return 4; //返回 4, 非以上三种情况
}

```


实验记录:

一、框架设计

1. 算法实现

1. 棋盘为二维的, 因此通过二维数组承载生命体, 令0为死, 1为生。
2. 因要实现棋盘大小的可输入, 而二维数组主函数定义时大小一定, 因此在主函数中定义足够大二维数组, 并利用函数形参传址调用确定二维数组大小及初值。

3. 因每一秒的数组都要输出, 故定义输出函数实现输出

4. 因每一秒的数组相较前1s都要更新, 故定义更新函数遍历上一秒二维数组, 判断出每点相邻的点数并保存在对应新定义的等大数组中, 再遍历二维数组, 依据相邻点数对每个元素赋新值。

(遍历数组 → 判断相邻点数 → 依据点数遍历数组赋新值)

5. 判断最终结果, 因结果有四种情况, 可设置返回值依据条件不同为1, 2, 3, 4的函数, 在主函数中利用 switch 输出最终情况。

6. 将游戏规则用伪代码表示, 并作为更新数据的依据:

if 生命体是活的,

1. 若周围生命体 > 3 , 死

2. 若周围生命体 $= 2$ 或 3 生

3. 若周围生命体 $= 0$ 或 1 死

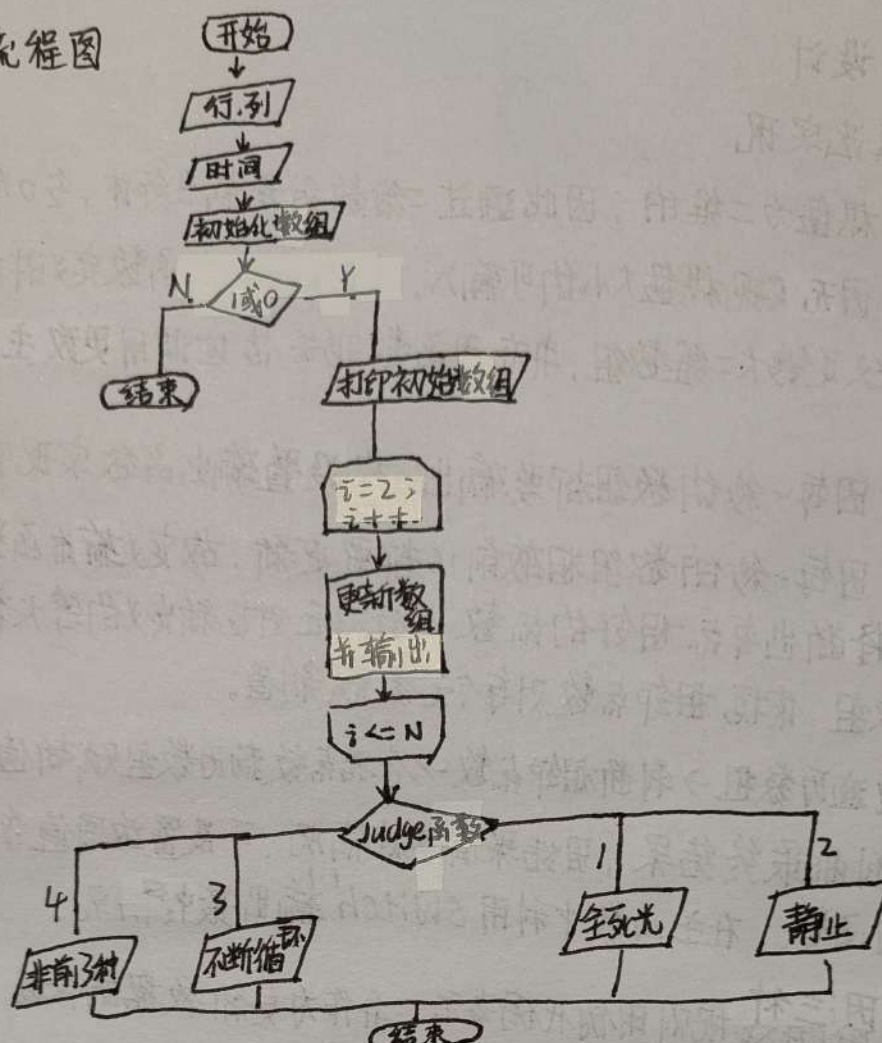
else 生命体是死的,

1. 若周围生命体 $= 3$ 生

2. 若为其他个数 死

实验记录:

12) 程序流程图



13) 各模块间关系: 每个模块功能由对应函数实现

1. 主函数: 负责读入数组大小及生存时间并判断初始数组的合法性。
负责调用功能函数实现特定功能
负责显示最终属于四种情况中的哪种
2. get 函数: 负责初始化数组即提供第1s时的数组。
3. put 函数: 负责输出对应时间下的数组。
4. update 函数: 负责判定当前1s数组中各元素相邻元素个数, 并据此更新数组^{统一}
5. judge 函数: 负责判断其在Ns之前全死亡或静止或循环或非前3种情况
6. transform 函数: 负责将每秒产生的数据依次记录进一维数组中, 以便后^{数组}续进行不同4秒之间数据的判关系判定。

二、详细设计

(1) 程序代码的详细注释：见代码在释

(2) 阐述关键部分代码及功能实现：

1. update 函数功能：① 记录上一秒数组各元素其相邻的元素个数：

实现方法：定义 `int geshu = 0`；负责记录某个元素的相邻的元素个数，
通过判断该元素周围八个元素是否存在，并且为1，若满足两者则 `geshu++`
并设置一个与棋盘等大的数组，在新数组对应位置记录下该元素周围元素个数。并通过循环遍历数组元素，实现对所有元素对应 `geshu` 的记录。

② 赋新值：通过区分原本元素为1或0两种情况，并根据①中记录的 `geshu`，结合游戏规则，遍历元素赋新值。

2. transform 函数功能：将每秒产生的数组元素依次排入全局变量的一维数组中。

目的：因每秒更新数组后其前1秒数据被覆盖，若不记录则无法进行某两秒之间的比较，无法判断最终结果。

3. judge 函数功能：实现最终结果的反馈：`return 1` 表示数组全为0

`return 2` 表示达到静止状态。

`return 3` 表示循环。

`return 4` 表示非以上三种情况。

功能实现：① 数组全为0：遍历二维数组，记录下0元素个数，若0个数=元素总个数，则 `return 1`，跳出 `judge` 函数

② 静止状态：因每秒数组已记录进一维数组，则从第一秒数组开始若其第一个元素等于第二秒数组第一个元素，则比较第二个元素依次类推，若不等跳出循环，开始比较第二秒数组第一个元素与第三秒数组，依次类推，直到找到某相邻2s数组同或遍历完仍不同，若找到 `return 2`，跳出循环。

③ 循环状态：与静止状态类似，只不过找出两个不相邻秒对应数组相等，第1秒比第3秒数组，若同则循环，若不同则第1s比第4s至最后1s，若仍不等则第2秒比第4s，以此类推 `return 3`

④ 非1,2,3状态则为第4种状态 `return 4`

实验记录:

13) 详细运行结果

① 程序调试: 通过编译时提示的错误修正部分语法错误, 运行时结果不符合预期, 则注释掉其他不相关内容, 单独修改调试相关部分, 修正逻辑错误。

② 测试结果: 调试完成后结果正确符合预期。

14) 调试过程:

① 开始时化二维为一维定义一维数组通过形参赋值, 但无法正确判断相邻个数。

原因: $\begin{matrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{matrix}$ `a[i][j]` 左侧本是没有相邻的, 但一维数组中 `&a[i][j]` 前面是 `&a[i][j-1]`

解决办法: 改用二维数组。

② 形参二维数组必须定义列的大小, 而题目要求输入行和列 ^{指定}

原因: C语言语法要求

解决办法: 定义足够宽的列, 并传递给形参实际的列, 用多少取多少。

③ 更新下一秒的数组不正确

原因: 一开始是判断前一个元素相邻个数后直接将其值更改, 从而影响下一个元素相邻个数的判断

解决办法: 先遍历数组, 记录下每个元素相邻元素的个数后统一更改元素数值。

④ 一开始 `judge` 函数中判断出哪种情况后 `break`, 实际结果错误。

原因: `break` 只能跳出内层循环, 但外面还有一层循环。

解决办法: 去掉 `break`, 采用 `return`, 直接返回一个特定值。

⑤ 未显示本应显示的数据

原因: 因循环中有逻辑错误, 使其未跳出循环。

解决办法: 注释掉其它内容, 修改 `for` 循环, 重新调试。

⑥ 在函数中赋了新值的变量, 对应变量在主函数中未改变

原因: 采用了传值调用

解决办法: 改为传址调用

实验记录:

① 定义的指针类型与赋给他值的指针类型不同

如 `int*` 型不能被 `int*[200]` 型赋值

解决办法: 更改指针类型

② 输入非0和1的数据不能得到有效反馈

解决办法: 初始化后遍历数组看是否全为1, 0, 若不是提示输入非法.

③ 在判断数组元素是否全为0时, 不知道如何才能表示全部元素为0.

解决办法: 有一个元素为0, 定义 `int c`, 令 `c++`; 如果最终 `c = 元素总个数`, 则全为0

④ 最终结果循环这一情况不知如何判断.

解决办法: 多次尝试下采用第1s数据比较第3s若不相等比较第4s至第N秒全不相等, 则第2s比第4s, 以此类推, 最终实现判断

⑤ 存在忘记声明函数, 变量定义重复, 语句无法实现有效判断, `scanf` 中忘记 "&", 没有关闭上一次程序等问题.

解决办法: 依据报错更改语法错误.

实验数据整理:

DA编程文件\生命游戏再修改.exe

请输入棋盘的行和列: 3 3
 请输入生存时间: 4
 请初始化棋盘, 生存输入1, 死亡输入0

```
0 1 0
0 1 0
0 1 0
```

第1秒的状态

```
0 1 0
0 1 0
0 1 0
```

第2秒的状态

```
0 0 0
1 1 1
0 0 0
```

第3秒的状态

```
0 1 0
0 1 0
0 1 0
```

第4秒的状态

```
0 0 0
1 1 1
0 0 0
```

最终开始产生循环

Process exited after 10.52 seconds

DA编程文件\生命游戏再修改.exe

请输入棋盘的行和列: 3 3
 请输入生存时间: 4
 请初始化棋盘, 生存输入1, 死亡输入0

```
1 0 1
0 1 0
1 0 1
```

第1秒的状态

```
1 0 1
0 1 0
1 0 1
```

第2秒的状态

```
0 1 0
1 0 1
0 1 0
```

第3秒的状态

```
0 1 0
1 0 1
0 1 0
```

第4秒的状态

```
0 1 0
1 0 1
0 1 0
```

最终不再变化

Process exited after 10.51 seconds

DA编程文件\生命游戏再修改.exe

请输入棋盘的行和列: 3 3
 请输入生存时间: 4
 请初始化棋盘, 生存输入1, 死亡输入0

```
1 0 0
0 0 1
1 0 0
```

第1秒的状态

```
1 0 0
0 0 1
1 0 0
```

第2秒的状态

```
0 0 0
0 1 0
0 0 0
```

第3秒的状态

```
0 0 0
0 0 0
0 0 0
```

第4秒的状态

```
0 0 0
0 0 0
0 0 0
```

最终全死光

Process exited after 9.052 seconds

报告

(注: 由于空间因素, 全部选择 3x3 棋盘 和生存 4s)

实验结论及建议:

结论: 实现生命游戏的编程要熟练掌握二维数组、指针、函数、局部变量与全局变量、循环等的知识。

最终生命游戏会生成四种结果: 全为0, 全相同, 循环, 不循环不相同不为0

建议: 优化: ① 可以尝试使用二维数组盛装每秒数据, 每行代表1s的数据, 这样比较不同时间数组时更为方便易懂。

② 可以利用指针在 judge 函数 return 返回前带回一个对应的秒数。这样就可以显示第几秒开始为0或静止或循环。